

AI EXTENSION IN PLAYWRIGHT / TYPESCRIPT

Prompt :

In ASK mode :

Prompt : Analyse the entire codebase and understand the implementation for Java/Selenium page class generation logic.

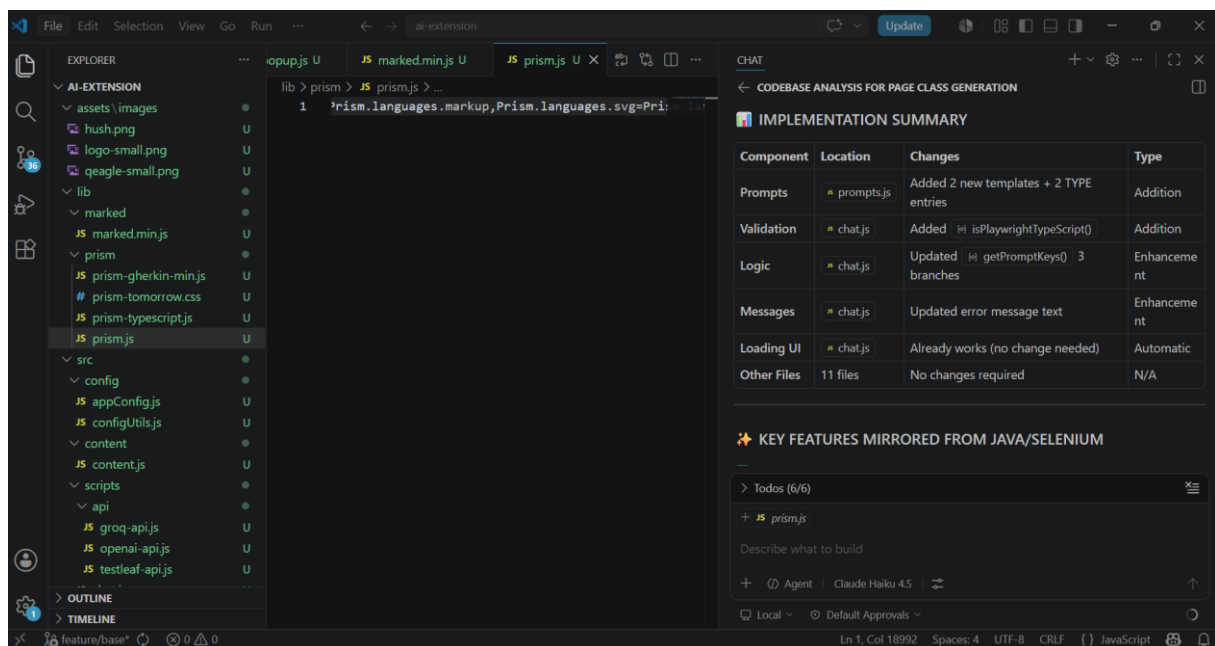
Extend the extension's capabilities to generate equivalent page classes for Playwright and TypeScript.

This new option should mirror the functionality currently available for Java/Selenium.

List all the impacted files.

Prompt 2 :

Agent mode : Step by Step, Implement for Playwright/TypeScript, with the exact same logic that is present for Java selenium



TypeScript and Playwright Selected :

AI Browser Extension for Code Generation

Code Generator

Settings

Supported Language

TypeScript

Browser Engine

Playwright

Select LLM Provider

Groq

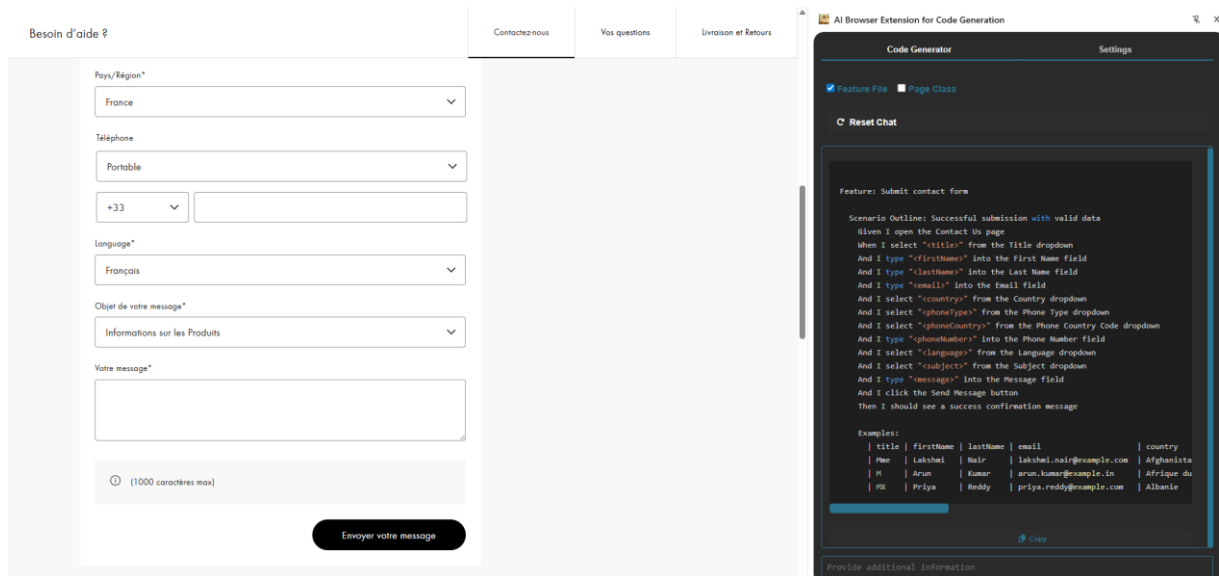
Select Model

openai/gpt-oss-120b

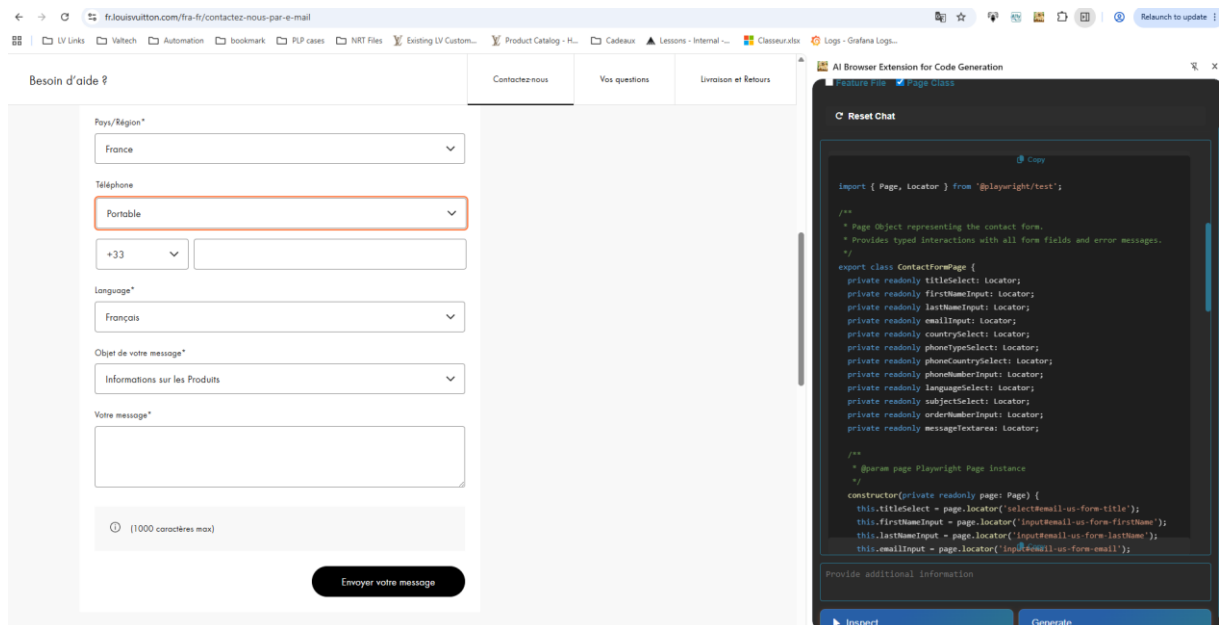
Groq API Key

.....

Feature File Only :



Page Class Only Selected:



Feature file and Page Class Selected :

The image shows a contact form on the left and a code editor on the right. The form has fields for Country/Region (France), Phone Type (Portable), Phone Country Code (+33), Language (Français), Subject (Informations sur les Produits), and a Message field. A 'Envoyer votre message' button is at the bottom. The code editor shows the Page Class for the contact form, with a table of test data.

Form Fields:

- Pays/Région*: France
- Téléphone: Portable
- +33
- Language*: Français
- Objet de votre message*: Informations sur les Produits
- Votre message*: (1000 caractères max)
- Envoyer votre message

Page Class Code:

```
import { Page, Locator } from '@playwright/test';

/**
 * Page Object representing the contact form.
 * Provides typed interactions with all form fields and error messages.
 */
export class ContactFormPage {
  private readonly titleSelect: Locator;
  private readonly firstNameInput: Locator;
  private readonly lastNameInput: Locator;
  private readonly emailInput: Locator;
  private readonly countrySelect: Locator;
  private readonly phoneTypeSelect: Locator;
  private readonly phoneCountryCodeSelect: Locator;
  private readonly phoneNumberInput: Locator;
  private readonly languageSelect: Locator;
  private readonly subjectSelect: Locator;
  private readonly messageInput: Locator;
  private readonly submitButton: Locator;

  constructor(page: Page) {
    this.titleSelect = page.getByText('title');
    this.firstNameInput = page.getByText('firstName');
    this.lastNameInput = page.getByText('lastName');
    this.emailInput = page.getByText('email');
    this.countrySelect = page.getByText('country');
    this.phoneTypeSelect = page.getByText('phoneType');
    this.phoneCountryCodeSelect = page.getByText('phoneCountry');
    this.phoneNumberInput = page.getByText('phoneNumber');
    this.languageSelect = page.getByText('language');
    this.subjectSelect = page.getByText('subject');
    this.messageInput = page.getByText('message');
    this.submitButton = page.getByText('submitButton');
  }

  async fillForm(
    title: string,
    firstName: string,
    lastName: string,
    email: string,
    country: string,
    phoneType: string,
    phoneCountryCode: string,
    phoneNumber: string,
    language: string,
    subject: string,
    message: string
  ): Promise<void> {
    await this.titleSelect.selectOption(title);
    await this.firstNameInput.fill(firstName);
    await this.lastNameInput.fill(lastName);
    await this.emailInput.fill(email);
    await this.countrySelect.selectOption(country);
    await this.phoneTypeSelect.selectOption(phoneType);
    await this.phoneCountryCodeSelect.selectOption(phoneCountryCode);
    await this.phoneNumberInput.fill(phoneNumber);
    await this.languageSelect.selectOption(language);
    await this.subjectSelect.selectOption(subject);
    await this.messageInput.fill(message);
    await this.submitButton.click();
  }
}
```

title	firstName	lastName	email	country
M	Arun	Kumar	arun.kumar@example.com	Afghanistan
Mme	Lakshmi	Nair	lakshmi.nair@example.in	Afrique du Sud
MR	Sanjay	Reddy	sanjay.reddy@example.com	Albanie

Feature: Submit contact form

Scenario Outline: Successful submission of the contact form with valid data

Given I open the contact form page

When I select "<title>" from the Title dropdown

And I type "<firstName>" into the First Name field

And I type "<lastName>" into the Last Name field

And I type "<email>" into the Email field

And I select "<country>" from the Country dropdown

And I select "<phoneType>" from the Phone Type dropdown

And I select "<phoneCountry>" from the Phone Country Code dropdown

And I type "<phoneNumber>" into the Phone Number field

And I select "<language>" from the Language dropdown

And I select "<subject>" from the Subject dropdown

And I type "<message>" into the Message field

And I click the Submit button

Then I should see a success confirmation message

Examples:

```
| title | firstName | lastName | email | country | phoneType |  
phoneCountry | phoneNumber | language | subject | message  
|
```

```
| M | Arun | Kumar | arun.kumar@example.com | Afghanistan | Portable | +1  
(US) | 9876543210 | Deutsch | Informations sur les Produits | Bonjour, je souhaite  
obtenir des informations sur les produits disponibles. |
```

```
| Mme | Lakshmi | Nair | lakshmi.nair@example.in | Afrique du Sud | Domicile |  
+1 (CA) | 9123456789 | Portugûês | Service Après-Vente | Bonjour, j'ai besoin  
d'aide pour la réparation de mon appareil. |
```

```
| MX | Sanjay | Reddy | sanjay.reddy@example.com | Albanie | Bureau | +7  
| 9988776655 | Östrich | Commandes en ligne | Bonjour, je souhaite passer une  
commande en ligne. |
```

Page class :

```
import { Page, Locator } from '@playwright/test';
```

```
/**
```

```
 * Page Object representing the contact form.
```

```
 * Provides typed interactions with all form fields and error messages.
```

```
*/
```

```
export class ContactFormPage {  
  private readonly titleSelect: Locator;  
  private readonly firstNameInput: Locator;  
  private readonly lastNameInput: Locator;  
  private readonly emailInput: Locator;  
  private readonly countrySelect: Locator;  
  private readonly phoneTypeSelect: Locator;  
  private readonly phoneCountrySelect: Locator;  
  private readonly phoneNumberInput: Locator;  
  private readonly languageSelect: Locator;
```

```
private readonly subjectSelect: Locator;

private readonly orderNumberInput: Locator;

private readonly messageTextarea: Locator;
```

```
/**
```

```
 * @param page Playwright Page instance
```

```
 */
```

```
constructor(private readonly page: Page) {
```

```
    this.titleSelect = page.locator('select#email-us-form-title');
```

```
    this.firstNameInput = page.locator('input#email-us-form-firstName');
```

```
    this.lastNameInput = page.locator('input#email-us-form-lastName');
```

```
    this.emailInput = page.locator('input#email-us-form-email');
```

```
    this.countrySelect = page.locator('select#email-us-form-country');
```

```
    this.phoneTypeSelect = page.locator('select#email-us-form-phone-  
    phoneNumberType');
```

```
    this.phoneCountrySelect = page.locator('select#email-us-form-phone-  
    phoneNumberCountry');
```

```
    this.phoneNumberInput = page.locator('input#email-us-form-phone-phoneNumber');
```

```
    this.languageSelect = page.locator('select#email-us-form-language');
```

```
    this.subjectSelect = page.locator('select#email-us-form-subject');
```

```
    this.orderNumberInput = page.locator('input#email-us-form-orderNumber');
```

```
    this.messageTextarea = page.locator('textarea#email-us-form-text');
```

```
}
```

```
/** Select a title (e.g., "Mme", "M", "MX") */
```

```
async selectTitle(value: string): Promise<void> {
```

```
    await this.titleSelect.selectOption({ value });
```

```
}
```

```
/** Fill the first name field */
```

```
async fillFirstName(firstName: string): Promise<void> {  
    await this.firstNameInput.fill(firstName);  
}
```

```
/** Fill the last name field */
```

```
async fillLastName(lastName: string): Promise<void> {  
    await this.lastNameInput.fill(lastName);  
}
```

```
/** Fill the e-mail field */
```

```
async fillEmail(email: string): Promise<void> {  
    await this.emailInput.fill(email);  
}
```

```
/** Select a country by its option value (e.g., "FRA", "US") */
```

```
async selectCountry(value: string): Promise<void> {  
    await this.countrySelect.selectOption({ value });  
}
```

```
/** Select phone type (e.g., "mobile", "home", "work") */
```

```
async selectPhoneType(value: string): Promise<void> {  
    await this.phoneTypeSelect.selectOption({ value });  
}
```

```
/** Select phone country code (e.g., "CA", "US", "RU") */
```

```
async selectPhoneCountry(value: string): Promise<void> {  
    await this.phoneCountrySelect.selectOption({ value });  
}
```

```
}
```

```
/** Fill the phone number field */
```

```
async fillPhoneNumber(number: string): Promise<void> {  
    await this.phoneNumberInput.fill(number);  
}
```

```
/** Select a language by its option value (e.g., "FRA", "POR") */
```

```
async selectLanguage(value: string): Promise<void> {  
    await this.languageSelect.selectOption({ value });  
}
```

```
/** Select a subject by its option value */
```

```
async selectSubject(value: string): Promise<void> {  
    await this.subjectSelect.selectOption({ value });  
}
```

```
/** Fill the hidden order number field */
```

```
async fillOrderNumber(orderNumber: string): Promise<void> {  
    await this.orderNumberInput.fill(orderNumber);  
}
```

```
/** Fill the message textarea */
```

```
async fillMessage(message: string): Promise<void> {  
    await this.messageTextarea.fill(message);  
}
```

```
/**
```

* Retrieve the error message associated with a given field.

* @param fieldId The base id of the field (e.g., "title", "firstName")

* @returns The error text or null if not visible

*/

```
async getFieldError(fieldId: string): Promise<string | null> {  
  const errorLocator = this.page.locator(`#${fieldId}-error-message`);  
  if (await errorLocator.isVisible()) {  
    return await errorLocator.textContent();  
  }  
  return null;  
}  
}
```